Webrouter

An open standard for web context

We're developing Webrouter as an open standard for how AI agents use web tools across providers: search, extraction, research and browsing. It pairs a common interface with a modular router that recommends the methods, tools and providers for a task.



Illustration only. Webrouter is software, not a physical device.

Why a common standard

Agents work across vendors with different APIs and toolsets. Finding current sources, reading a known URL, interacting with a website and researching a question require different methods. Applications need a consistent way to call those tools and use their results, while choosing an approach that fits the task and its quality, latency and cost constraints.

Two layers

1. A common interface for execution. A shared request and result contract covers search, extraction, research and browsing through provider adapters. Applications can use this interface directly, without the routing layer. Available capabilities depend on the configured adapter.

2. A modular router for planning. The router recommends methods, tools, providers or a sequence of steps. The choice starts with how to approach the task, then which tools can carry it out. The host agent or runtime executes the recommended plan through the common interface.

Connect your agent

Use the Python library or connect the local MCP server to your agent. With private-repo access, clone the repository, run `make setup` and add your provider credentials. This example selects Nimble explicitly:

```
router = WebRouter(Provider.NIMBLE)
response = router.search("what did Datadog launch this week", max_results=5)
[r.content for r in response.results]  # same field, every backend
```

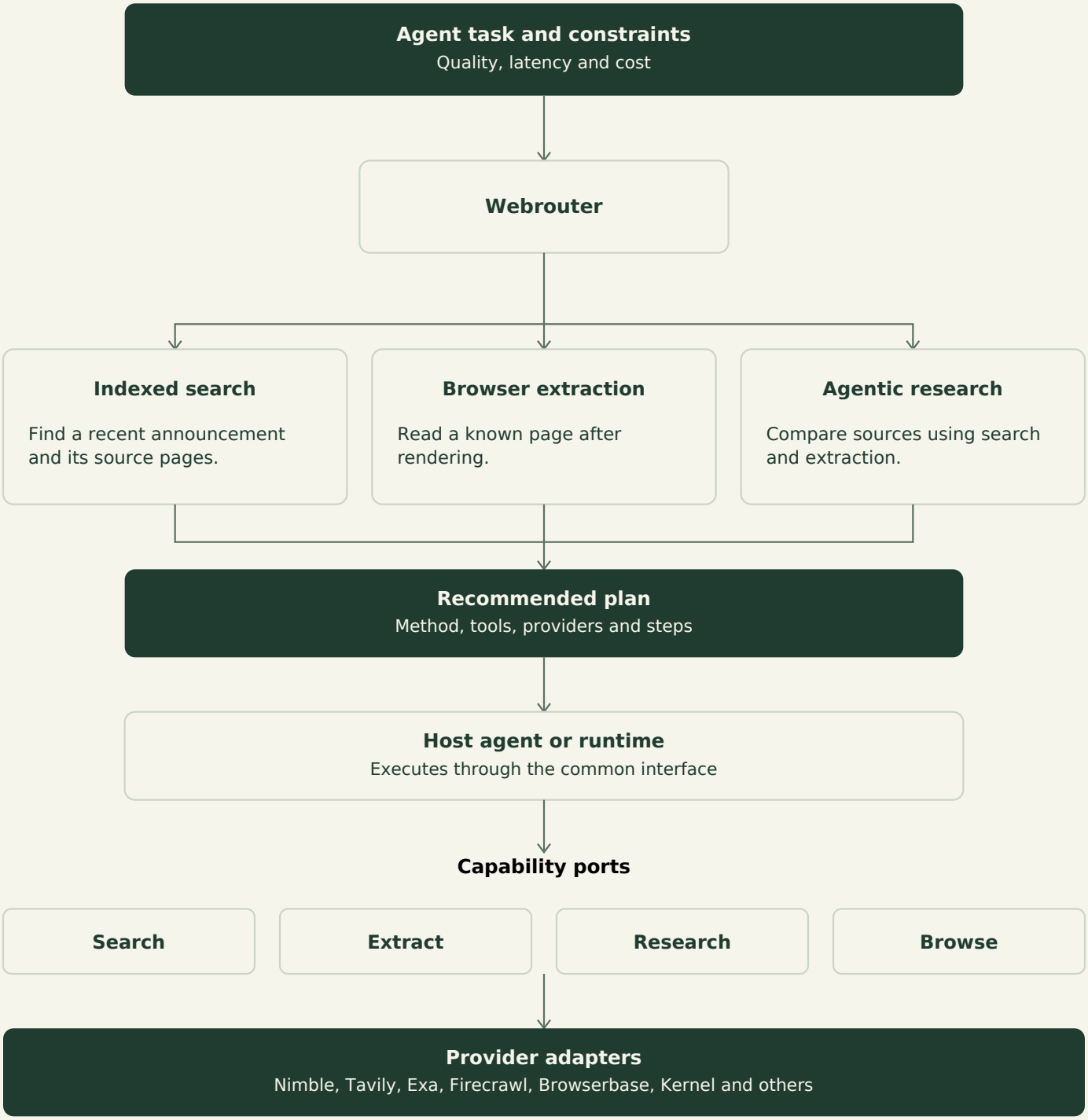
You can switch to Tavily or Exa using the same call and result format. This is direct interface use; routing is optional.

October 22 walkthrough. Imports, setup and credentials omitted.

Task-aware routing is still evolving, and preview APIs may change.

How routing works

Routing starts with the method, then the tools and providers. A plan can combine methods; the host agent or runtime executes the recommended steps.



Adapters normalize results; capabilities vary. Browse adds actions on a site.